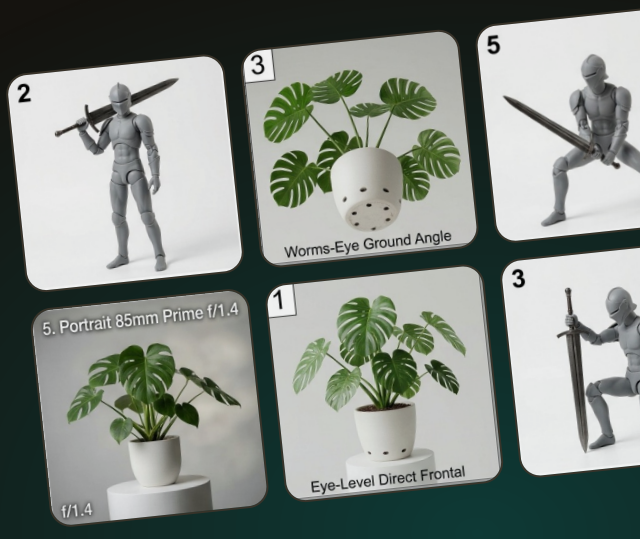
Prompt Forge

How we built it — a plain-English guide

A tour of the app, the code behind it, the 97-pose studio library, and the 27-angle camera gallery. Written so a high-school reader can follow it, with the real technical words kept in (and explained).

React 19 · TypeScript · TanStack Start · Tailwind v4 · Vite · PostgreSQL · 74 passing tests



Build guide · October 31, 2026

knowmxd / Prompt-Forge

START HERE

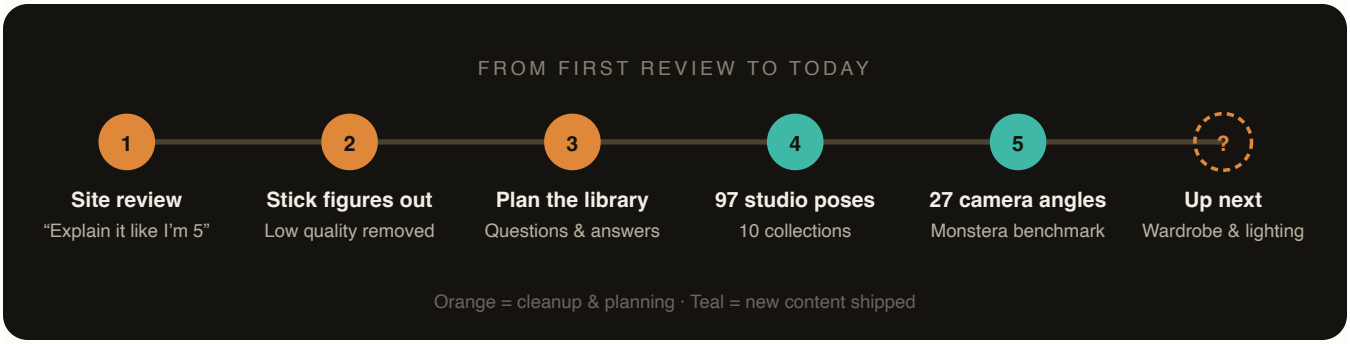
What’s inside

Eleven short chapters. You can read them in order or jump to the part you care about.

1. **What is Prompt Forge?** — the problem it solves, in one page
2. **The big picture** — paste → parse → edit → copy
3. **The toolbox** — the technologies and the map of the project folders
4. **The Prompt Blueprint** — the 15 slots every prompt gets sorted into
5. **The parser** — how messy text becomes neat structure
6. **The workbench** — the screen you actually use
7. **Where your stuff is saved** — browser, image vault, and cloud
8. **The Pose Library** — out with stick figures, in with 97 studio poses
9. **Camera & Angles** — 27 plant-powered lessons in perspective
10. **The “Looks” engine** — how one click edits your prompt
11. **How we know it works** — tests and quality gates

Plus: known rough edges, the roadmap, how to run the project yourself, and a glossary of every technical word.

Our journey so far



CHAPTER 1

What is Prompt Forge?

A workshop for building really good instructions for AI image generators.

When you ask an AI to make a picture, the words you type are called a **prompt**. A lazy prompt (“a knight on a cliff”) gets a lazy picture. A great prompt describes the *person*, the *clothes*, the *pose*, the *camera*, the *lighting*, the *mood*, and even what to *avoid*. That’s a lot to keep track of, and most people forget half of it.

> **THINK OF IT LIKE THIS**
> A prompt is a recipe. Prompt Forge is the recipe card with labeled boxes: *Ingredients*, *Oven temperature*, *Cooking time*. You can’t forget the oven temperature when there’s an empty box staring at you.

What it does, in four verbs

1 PASTE ANY MESSY PROMPT

2 IT SORTS THE WORDS INTO SLOTS

3 YOU EDIT OR CLICK IN READY-MADE LOOKS

…and then **4**, you copy the finished prompt out as clean **JSON** (a tidy, labeled text format) and paste it into your image tool.

What’s been added on top

97 STUDIO-QUALITY POSES IN 10 COLLECTIONS

27 CAMERA ANGLES, FRAMINGS AND LENSES

> **JARGON CHECK · JSON**
> JSON (say “JAY-son”) is a way of writing information as `"label": "value"` pairs, grouped inside curly braces. Computers love it because it’s predictable; humans can read it too. Example: `"lens": "85mm"`.

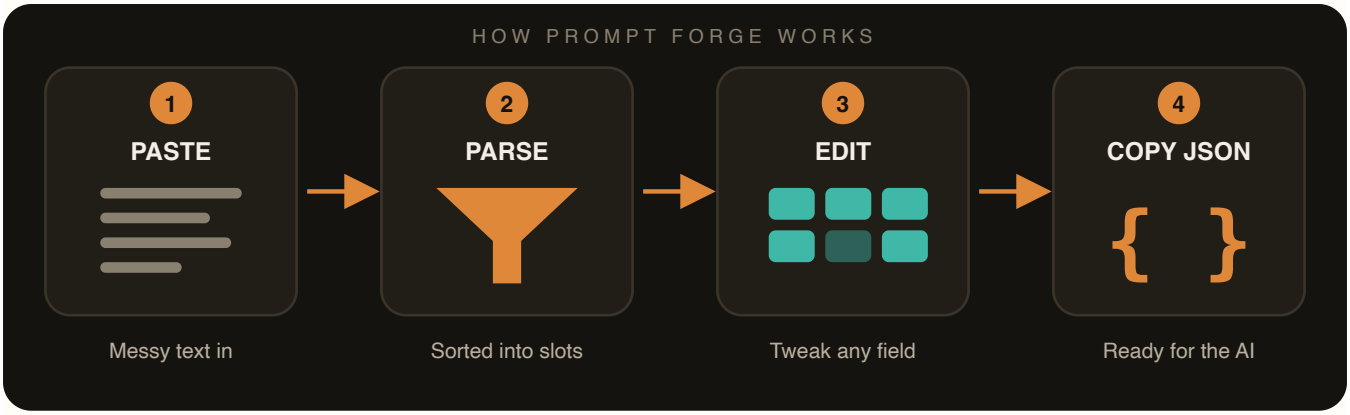
Who it’s for

Artists and photographers who make images with AI and want repeatable, high-quality results: character artists (knights, elves, rangers), portrait and fashion shooters, and anyone who wants a library of proven poses and camera setups to pick from instead of reinventing them every time.

CHAPTER 2

The big picture

Everything the app does fits into four steps.



The whole app in one picture.

Step by step

1. **Paste.** You drop any prompt into the *Source* box on the left. Paragraph, comma-separated tags, or existing JSON all work.
2. **Parse.** The app reads it and files each phrase in the right slot ("85mm lens" goes under *Optics*, "long silver hair" under *Hair*). *Parse* just means "read and organize."
3. **Edit.** Every slot is a text box in the middle of the screen. Fill empty ones, or click a ready-made **Look** (a pose, camera angle, light, or setting) from the right-hand shelf.
4. **Copy.** A live JSON preview at the bottom updates as you type. One button copies it.

> **JARGON CHECK · FRONTEND VS. BACKEND**
>
> The **frontend** is everything you see and click in the browser. The **backend** is code that runs on a server somewhere else, usually to save data. Prompt Forge is mostly frontend; the backend only wakes up when you sign in to save to the cloud.

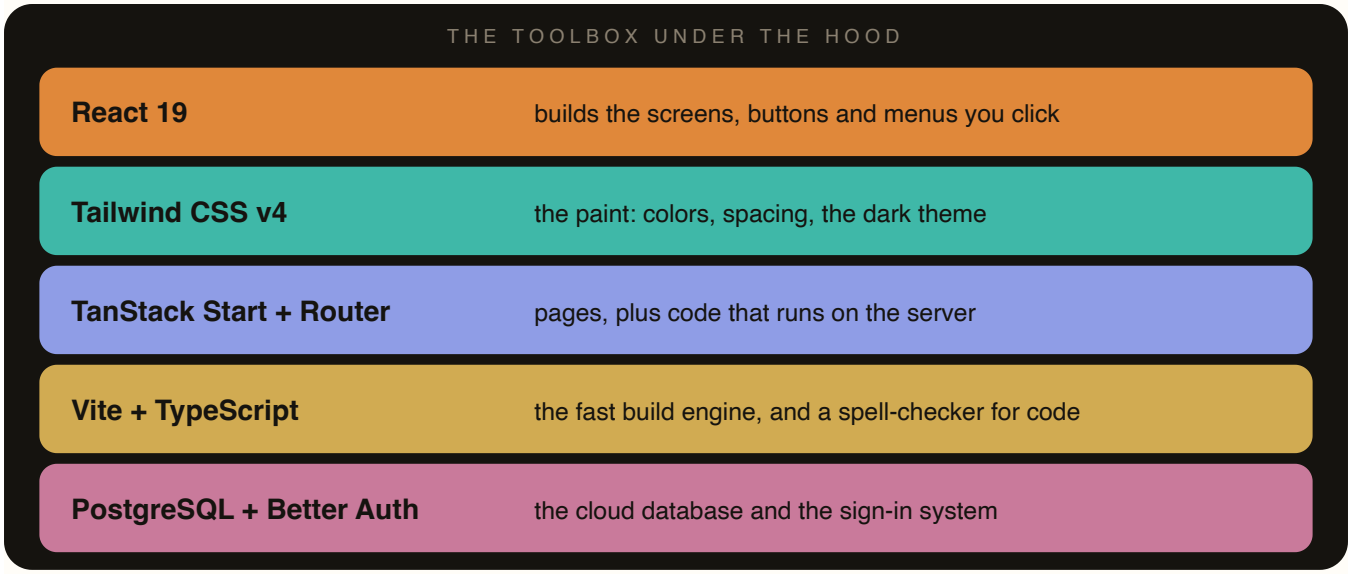
No AI inside the parser

An important detail: the parser doesn't call any AI service. It uses **rules** (keyword lists and pattern matching) so it's instant, free, works offline, and gives the same answer every time. The AI you'll eventually use is the image generator that receives your finished prompt.

CHAPTER 3

The toolbox

What the app is made of, from the screen you touch down to the database.



Top layer = what you see. Bottom layer = where saved data lives.

TOOL	IN PLAIN ENGLISH
React 19	A library for building screens out of small reusable pieces called **components**. A pose card is a component; so is the whole workbench.
TypeScript	JavaScript with labels on everything (“this value is text,” “this one is a list”). The computer checks the labels and catches mistakes before anyone sees them. That check is the `typecheck` step.
Tailwind CSS v4	Styling done by adding small class names (like `rounded-lg`) right on each element, instead of writing a separate stylesheet.
TanStack Start / Router	The framework that turns the app into real web pages, handles page addresses (routes), and lets us write **server functions**, code that runs safely on the server.
Vite	The engine that bundles all the code for the browser. In development it updates the page instantly while you edit (called **HMR**, hot module replacement).
PostgreSQL + Better Auth	A database for saved work and a ready-made sign-in system. Only used if someone signs in.

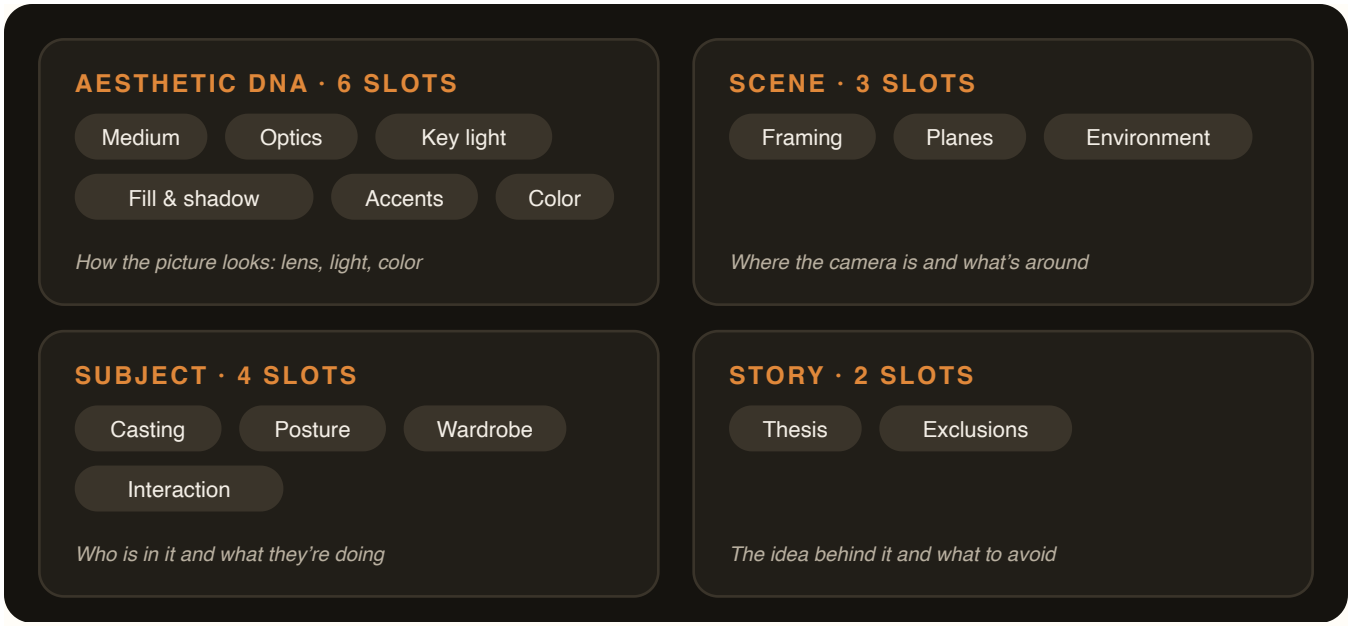
A map of the project

WHERE	WHAT LIVES THERE
`src/components/prompt-forge.tsx`	The big one: the whole workbench screen (about 2,000 lines).
`src/components/pose-catalog-modal.tsx`	The full-screen Pose Reference Gallery.
`src/components/camera-catalog-modal.tsx`	The full-screen Camera & Optics Gallery.
`src/components/pose-art.tsx`	The little card that shows one pose or angle thumbnail with its badge.
`src/lib/prompt/model.ts`	The blueprint: what a prompt looks like as data.
`src/lib/prompt/parse.ts` & `assemble.ts`	The sorting machine (parser) and the code that files results into the blueprint.
`src/lib/prompt/pose-database.ts`	All 97 poses: names, categories, descriptions, images.
`src/lib/prompt/camera-database.ts`	All 27 camera angles, framings and lenses.
`src/lib/prompt/looks.ts`	Combines poses, angles, lights, settings into one “Look” list and applies them.
`src/lib/prompt/storage.ts`, `cloud.ts`, `images.ts`	Saving: browser storage, cloud sync, and the photo vault.
`public/poses/`, `public/camera/`	The actual images (pose mannequins; plant camera cards).
`migrations/`	Instructions that create the database tables.
`*.test.ts` files	Automatic checks that the logic still works (74 tests).

CHAPTER 4

The Prompt Blueprint

Fifteen slots, in four groups. Every prompt gets sorted into the same shape.



The 15 "clusters" you see in the left-hand menu of the workbench.

Under the hood each slot is a field inside one big nested JSON object, defined in `model.ts` as a **type** called `PromptDoc`. Here's a trimmed view of that shape:

```
"aesthetic_dna": {
  "medium_format": "oil painting",
  "optics": { "lens_type_and_focal_length": "85mm", "aperture_and_dof": "f/1.4" },
  "lighting_architecture": { "key_source": {…}, "accent_and_practicals": {…} },
  "color_science": { "dominant_palette": ["#8A1C1C"] }
},
"scene_composition": { "framing_doctrine": {…}, "environmental_staging": {…} },
"subjects": [ { "subject_id", "casting_and_physique", "kinetic_posture",
"wardrobe_architecture"… } ],
"creative_thesis": {…},
"anti_cliche_exclusions": ["no text", "no watermark"]
```

> **THINK OF IT LIKE THIS**
> It's a filing cabinet with four drawers, and each drawer has folders. "Subjects" is a *list*, so you can have several characters in one scene, each with their own pose and wardrobe.

Why a strict shape matters

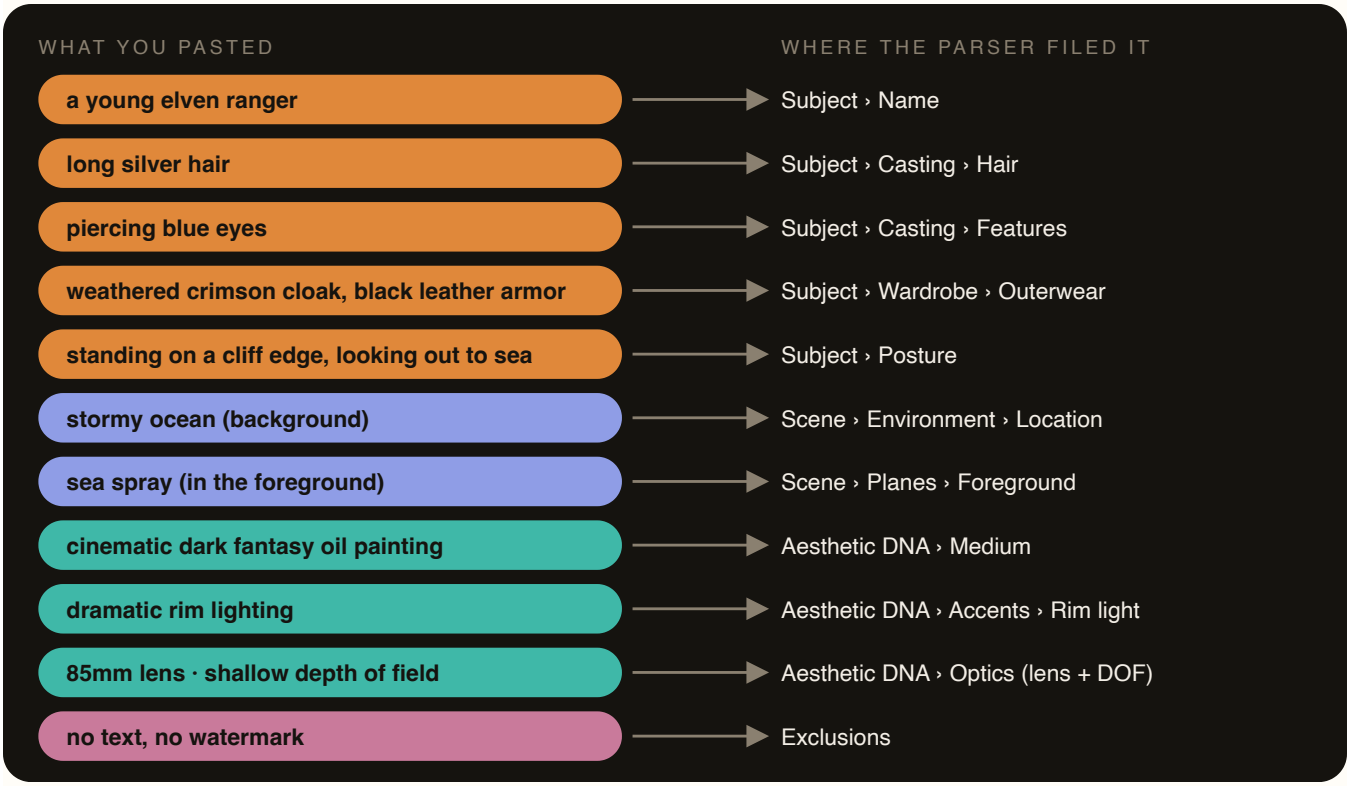
- **Nothing gets forgotten.** Empty slots are visible. The app shows "16 filled" style counters.
- **Looks can write to exact spots.** A pose card knows to write only into `kinetic_posture`; an angle card writes into framing and optics. They never overwrite each other.
- **Old data still works.** A `coerceDoc()` function repairs older or malformed saves into the current shape instead of crashing.

CHAPTER 5

The parser

The sorting machine that turns one messy sentence into 15 tidy slots.

Here's the sample prompt that ships in the app. We ran the real parser on it. The result below is its actual output, not a mock-up.



Orange = the person · Blue = the scene · Teal = the look · Pink = things to avoid.

How it decides

1. **Detect the format.** Is it JSON? Labeled lines (`Hair: silver`)? Plain sentences? The parser reports which mode it found (`json`, `labeled`, `freeform`) and says things like "Sorted this into 10 sections."
2. **Split into phrases** at commas, "and," "with," and similar joiners.
3. **Match keywords.** Lists of words route phrases: "lens, mm, f/" → Optics; "hair, braid, beard" → Hair; "armor, cloak, boots" → Wardrobe; "no …, without …" → Exclusions.
4. **Peel off special things.** Hex color codes like `#8A1C1C` go to the palette; Midjourney-style flags and aspect ratios get stripped out of the way.
5. **File it** into the Blueprint using `assemble.ts`.

JARGON CHECK · REGEX

A **regular expression** (“regex”) is a tiny pattern language for matching text. For example, `/\b(hair|braid|beard)/i` means “find any of these words, ignoring capital letters.” The parser is full of them.

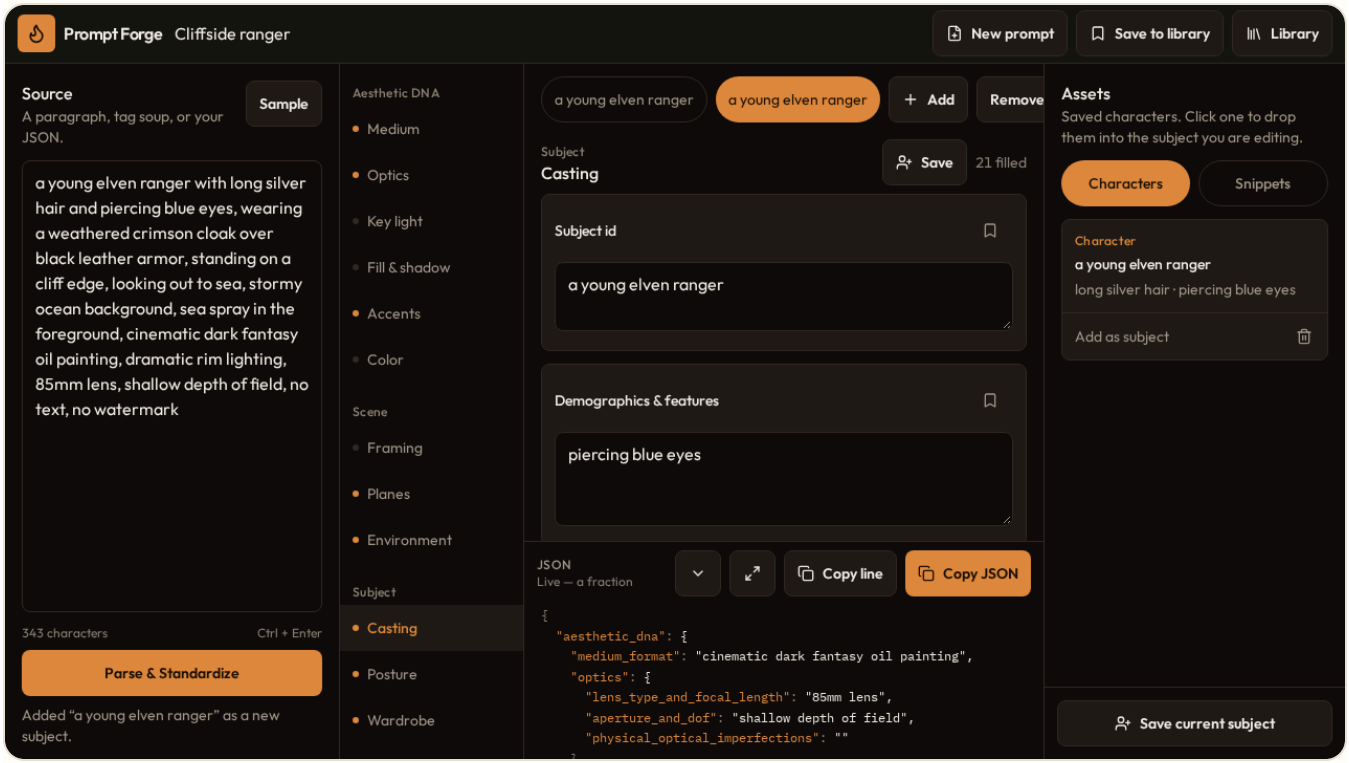
HONEST LIMITS

Rules can’t understand meaning the way an AI can. Phrases that don’t contain a known keyword may land in a general slot, so a quick glance and tweak after parsing is part of the workflow. In the sample above, “weathered” was also copied to *Wear & patina* and “leather” to *Textiles*. That’s the keyword rules doing their best.

CHAPTER 6

The workbench

The one screen where everything happens.



Left: your source text. Center-left: the 15-slot menu. Center: editable fields and the live JSON. Right: the Assets shelf.

The four zones

ZONE	WHAT YOU DO THERE
Source (far left)	Paste a prompt and press **Parse & Standardize** (or Ctrl+Enter). The **Sample** button loads the ranger example.
Cluster menu	The 15 slots, grouped under Aesthetic DNA, Scene, Subject, Thesis. A dot beside a name means it has content.
Editor + JSON	Text boxes for the open slot. Underneath is the live JSON, with **Copy line** and **Copy JSON** buttons and an expand button for full-screen reading.
Assets shelf (right)	Saved characters, snippets, bookmarks, and the **Looks** shelf with poses, camera angles, lights and settings.

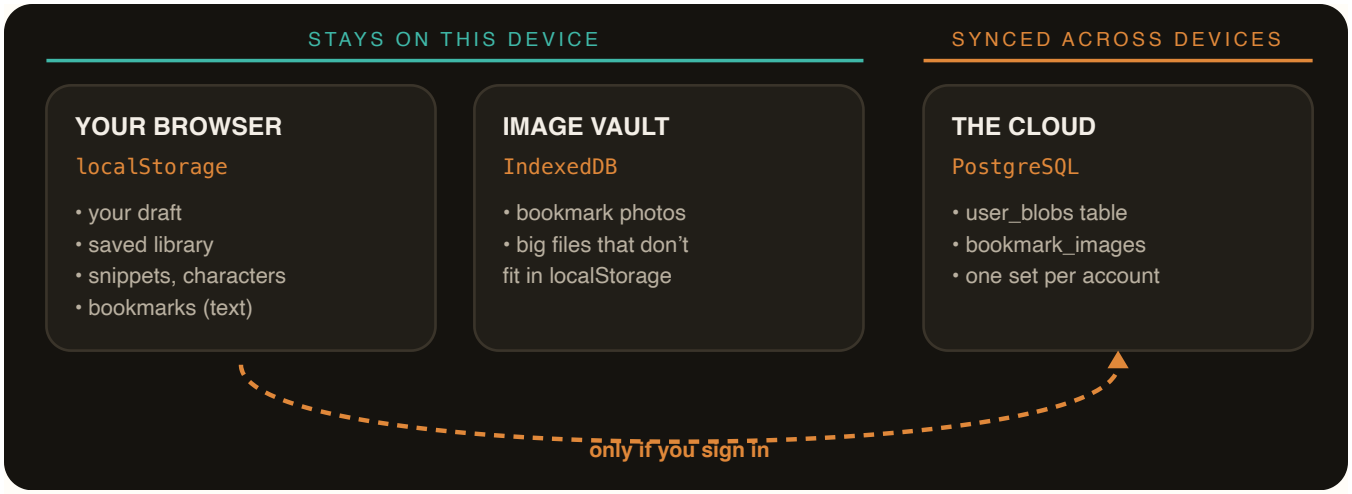
Handy features

- **Multiple subjects.** Tabs across the top let you add or remove characters. Each gets its own casting, posture and wardrobe.
- **Characters.** Save a finished subject and later drop it into any new prompt with “Add as subject.”
- **Snippets.** Reusable bits of text (“85mm, shallow depth of field”) you can insert anywhere.
- **Library.** Save whole prompts, search them, and reopen later.
- **Bookmarks.** Save a whole subject or just one section, add tags and a reference photo, and apply several at once.

CHAPTER 7

Where your stuff is saved

Three places, and only one of them needs an account.



Everything works offline. Signing in adds a copy in the cloud.

Why three places?

- **localStorage** is a small notepad built into every browser (about 5 MB). It's perfect for text but too small for photos.
- **IndexedDB** is a bigger browser database. Photos go there as **blobs** (raw file data).
- **The cloud** (PostgreSQL) is a real database on a server. A **server function** in `cloud.ts` checks who you are with `authMiddleware` and then reads or writes only *your* rows. The two tables are created by the file `migrations/0002_library.sql`.

> JARGON CHECK · MIGRATION & MIDDLEWARE
>
> A **migration** is a small script that creates or changes database tables, like a construction blueprint for the filing cabinet. **Middleware** is a checkpoint that runs before your code, here it's the bouncer that asks "who are you?" before letting a request touch saved data.

Safety nets

- Every read goes through a "coerce" function that cleans unexpected data, so a corrupted save can't break the app.
- Storage keys are versioned (`prompt-forge:library:v2`) so future changes can migrate old saves safely.
- Old field names (like `clothing` or `camera`) are translated to the new cluster names automatically.

CHAPTER 8

The Pose Library

Out with the stick figures. In with 97 studio-quality poses.

The problem

The first version drew poses as tiny wobbly stick figures. They were hard to read and looked cheap, and a pose reference is only useful if you can actually *see* the pose. So we removed them completely and started over with a different goal: a big, well-organized library that feels like a photo-studio reference book.



NEW: neutral matte mannequin. Shape and weight are clear.



OLD: stick figure (illustration). Removed.

How we planned it

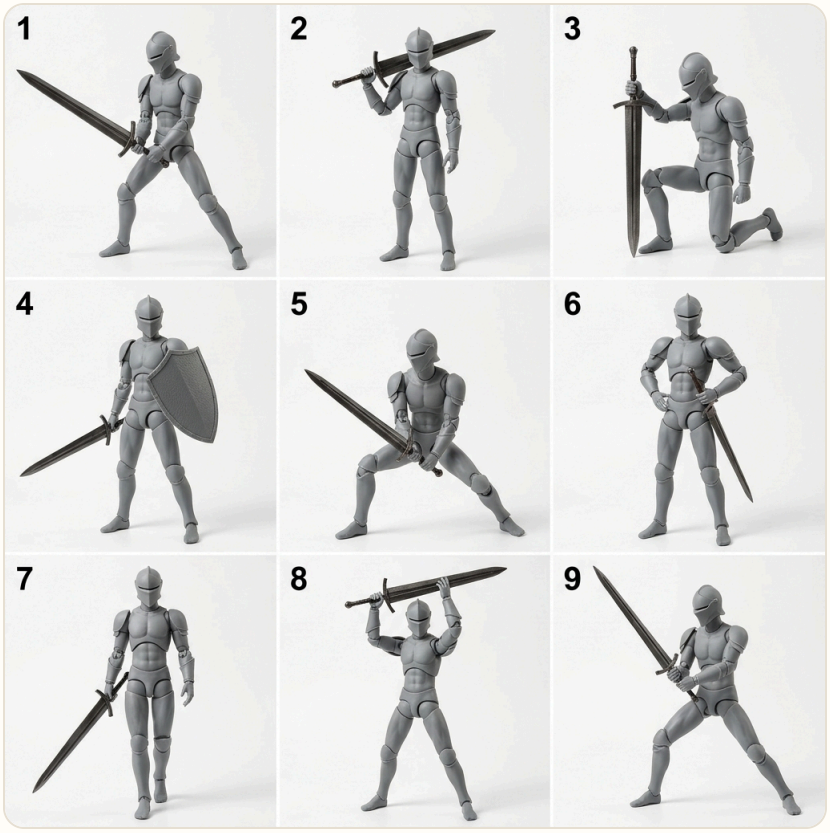
Before building, we asked a round of questions and you answered them. The decisions that shaped everything:

- **Use existing-style imagery, not hand-drawn art.** Easiest for us to produce and the quality is much higher.
- **Organize for real use.** Not just standing/sitting, but by *subject*: knights & warriors, fashion & photo shoots, fine-art paintings, cinematic scenes, couples, and so on.
- **Quality over quantity,** with hundreds as the long-term goal. We shipped 97 strong ones first.

- **Subject-specific poses are a later step.** (A knight's pose won't suit a portrait shoot.) For now, categories carry that meaning.

How the images were made

1. **Generate a contact sheet.** One AI image shows a 3×3 grid of nine poses of the same neutral grey mannequin on a white background.
2. **Slice it.** A script cuts each sheet into nine separate JPEGs in `public/poses/`.
3. **Describe each pose in words.** Every card gets a written `posture` sentence, because *that sentence* is what goes into the prompt.
4. **Register it in the database file** with an ID, category, stance and search tags.

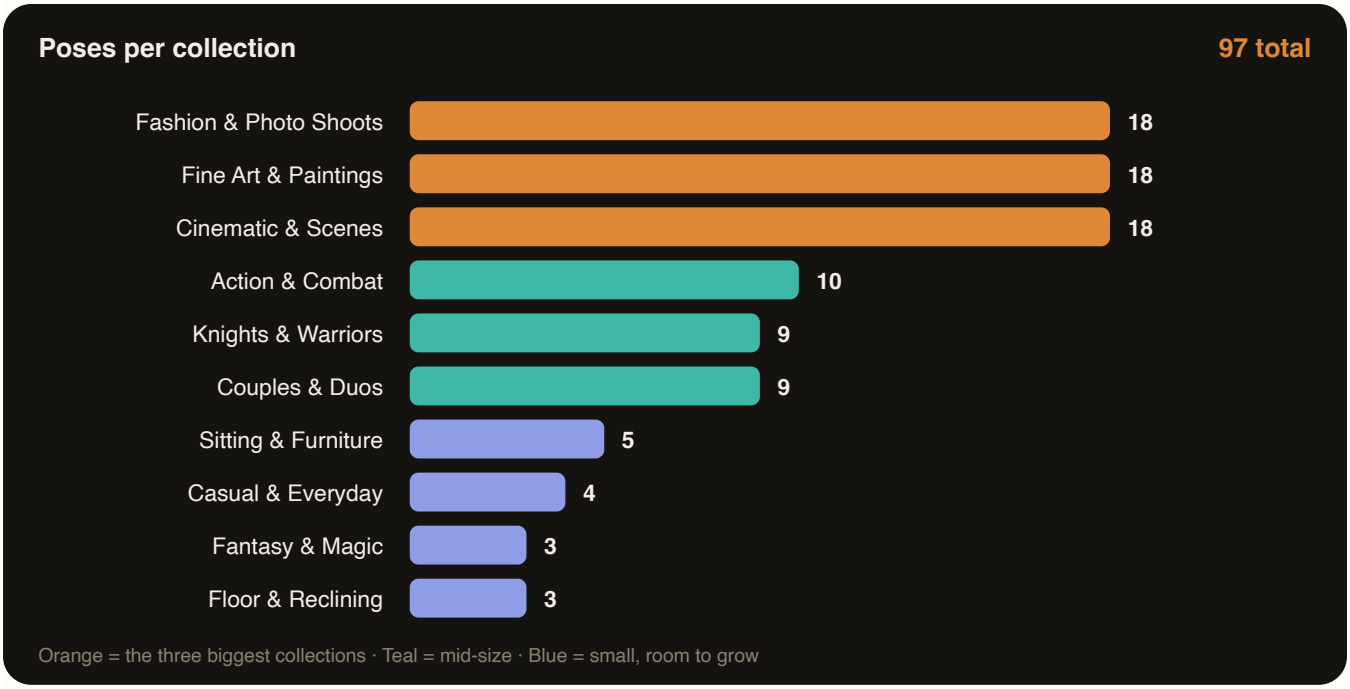


One generated contact sheet (Knights & Warriors) before slicing into nine cards.

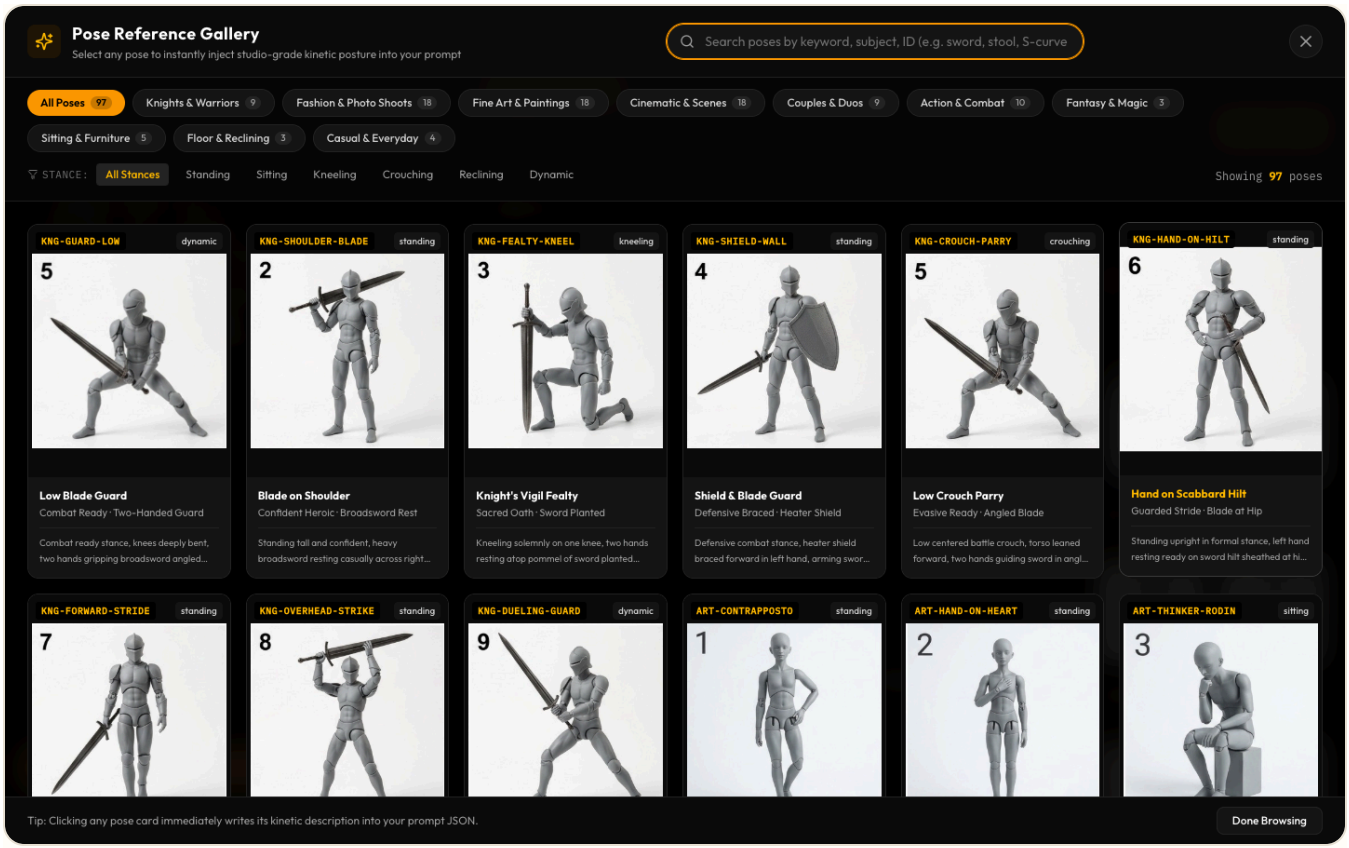
What one pose looks like as data

```
{
  id: "kng-shoulder-blade",
  name: "Blade on Shoulder",
  category: "knights",   // which collection
  stance: "standing",     // standing / sitting / kneeling …
  posture: "Standing tall and confident, heavy broadsword resting
           casually across right shoulder, left arm loose at side…",
  image: "/poses/pose_knight_02.jpg",
  tags: ["knight", "sword", "heroic" …]
}
```

The 97 poses



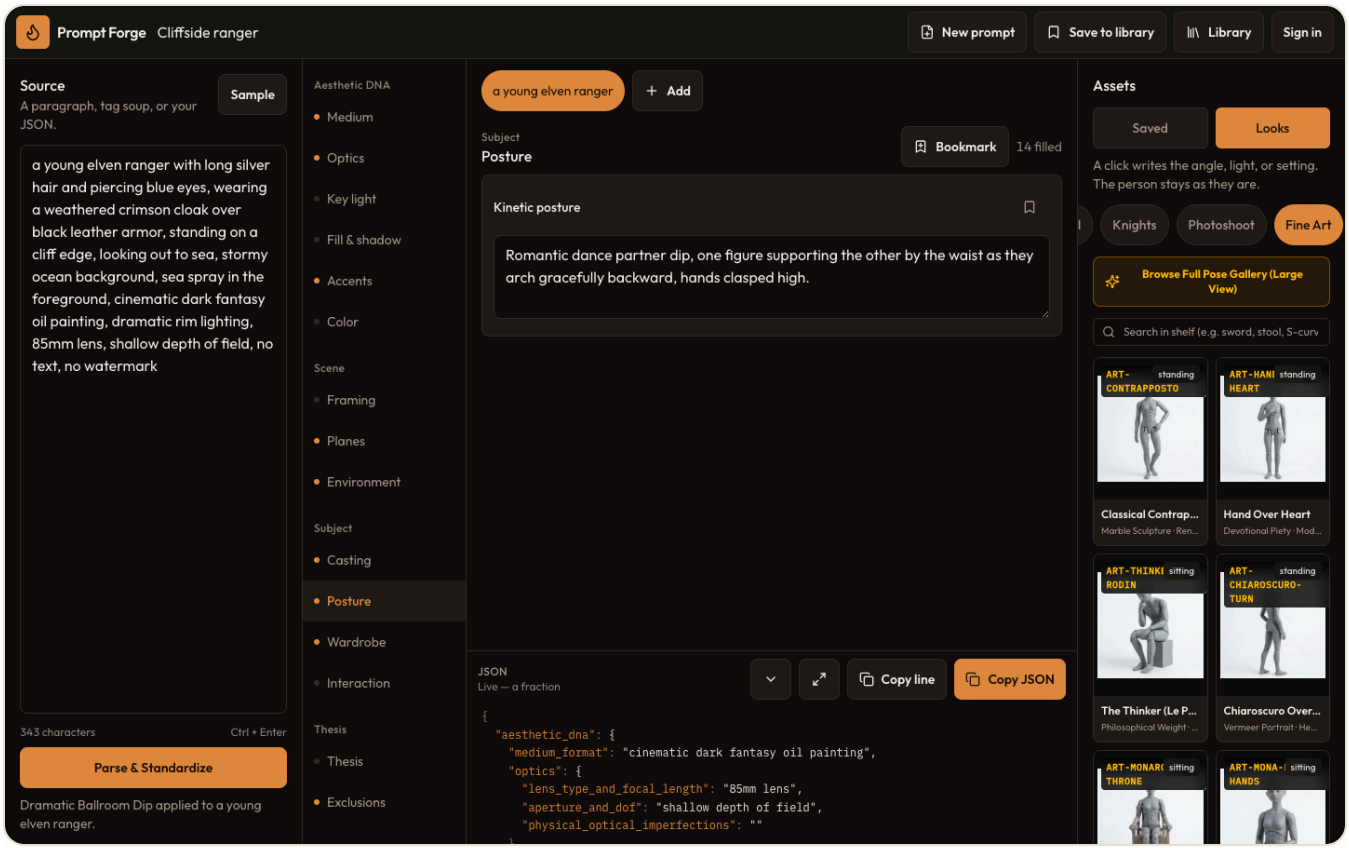
Where the library is deep, and where it needs more poses.



The Pose Reference Gallery: category pills with live counts, stance filters, and keyword search.

The gallery, feature by feature

- **Category pills** show a live count (`All Poses 97`, `Knights & Warriors 9`…).
- **Stance filters** narrow by Standing, Sitting, Kneeling, Crouching, Reclining or Dynamic.
- **Search** looks at name, ID, subtitle, description and tags, so “sword,” “stool” or “S-curve” all work.
- **Click a card** and it highlights, shows “Applied,” and the pose sentence is written into the open subject’s `kinetic_posture` field. Props listed on the pose (like “broadsword”) are merged into the scene’s props.

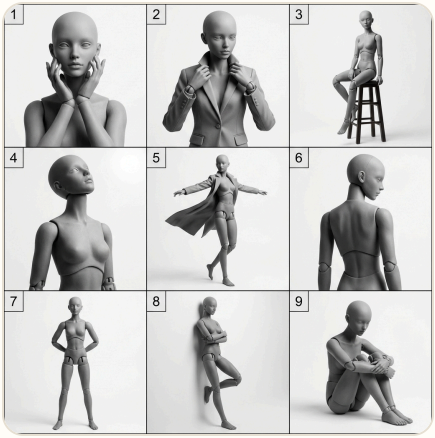


After clicking “Dramatic Ballroom Dip,” the exact wording appears in the Posture field and in the live JSON.

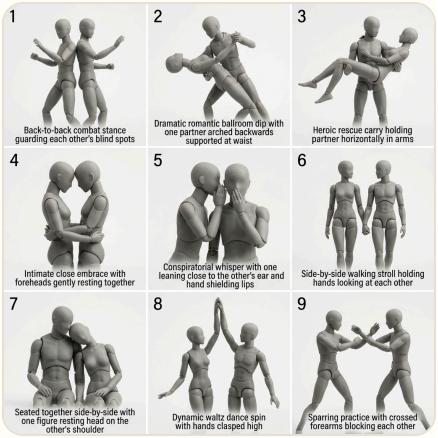
A tour of the collections



Fine Art: Rodin, Michelangelo, Botticelli…



Fashion & photo shoots



Couples & duos

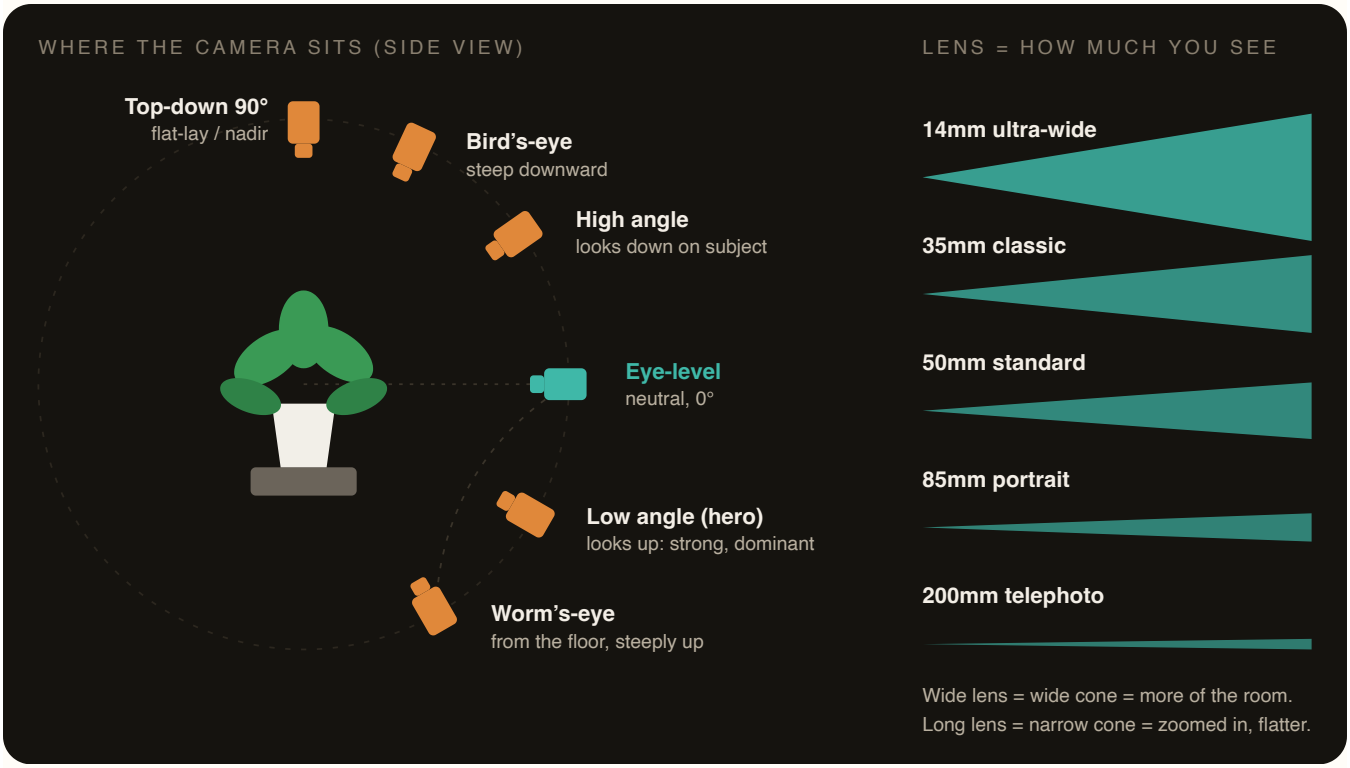
CHAPTER 9

Camera & Angles

27 options, all shown with the same plant, so the *angle* is the only thing that changes.

WHY A PLANT?

If every example used a different subject, you'd be comparing apples to oranges. A monstera in a white pot on a plain pedestal is the same in every picture, so when the view changes you instantly see what the camera did: the leaves stretch, the pot shrinks, the background melts away. It's a science-fair control experiment for photography.

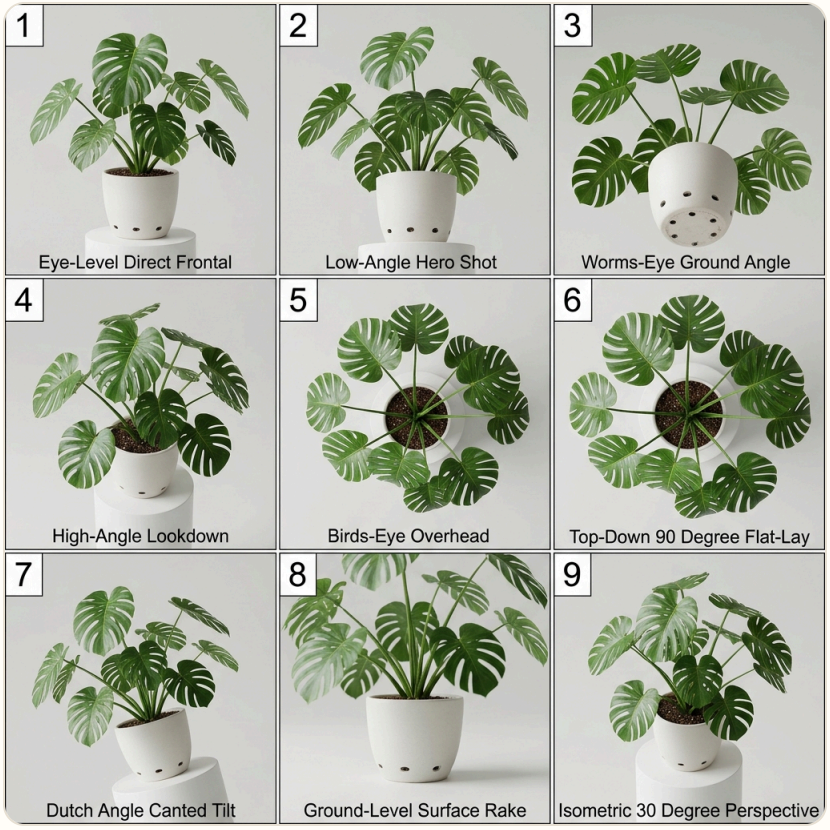


Positions are where the camera sits. The badge numbers on the cards (like "+60° WORM") describe how steeply it tilts.

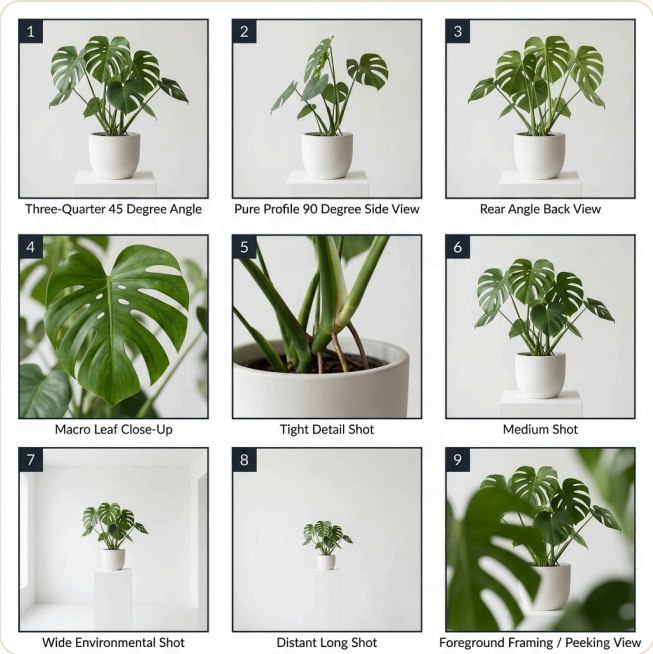
The 27 cards in three collections

9	9	9
ELEVATIONS & ANGLES	FRAMINGS & ROTATIONS	LENSES & DEPTH OF FIELD

COLLECTION	THE NINE CARDS
Elevations & angles height and tilt	Eye-level · Low-angle hero · Worm's-eye · High-angle · Bird's-eye · Top-down 90° · Dutch (tilted) · Ground-level skim · Isometric 30°
Framings & rotations turning and distance	Three-quarter 45° · Profile 90° · Rear 180° · Macro leaf · Tight close-up · Medium · Wide environmental · Distant long shot · Foreground peek
Lenses & depth of field zoom and blur	14mm ultra-wide · 24mm wide · 35mm classic · 50mm standard · 85mm portrait (f/1.4) · 135mm · 200mm super-tele · f/1.2 dreamy blur · f/16 everything sharp



Elevations & angles: same plant, nine camera positions.



Framings & rotations



Lenses & depth of field

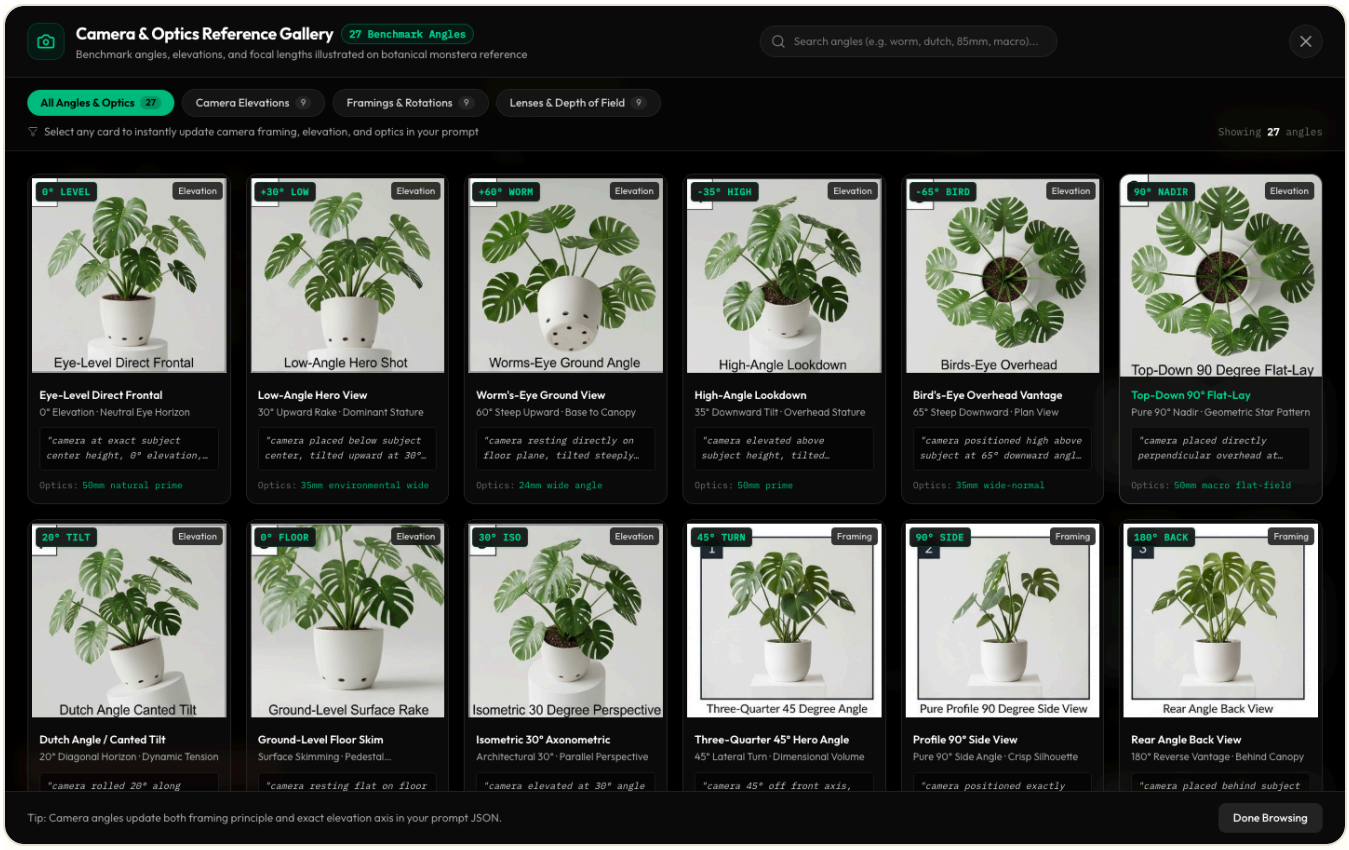
PHOTO WORDS, TRANSLATED

Focal length (mm): how "zoomed" the lens is. Small = wide view, big = zoomed in. **Aperture (f/number)**: how wide the lens opens; a small f-number (f/1.4) lets in lots of light and blurs the background. **Depth of field (DoF)**: how much of the picture is in focus. **Bokeh**: the soft, creamy blur. **Dutch angle**: tilting the camera so the horizon slants, for tension. **Nadir**: straight down.

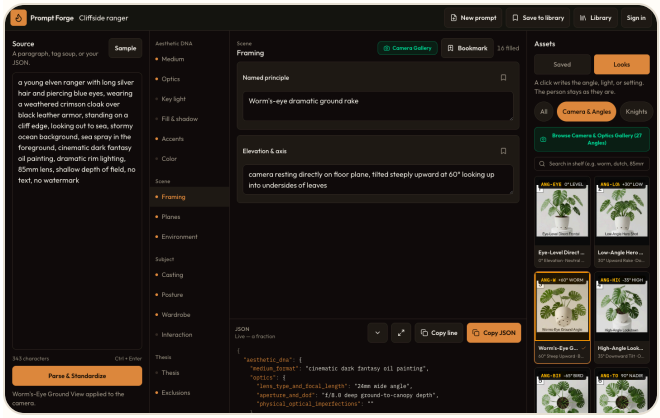
What clicking a card does

Each camera card carries four pieces of text, and it writes them into the prompt:

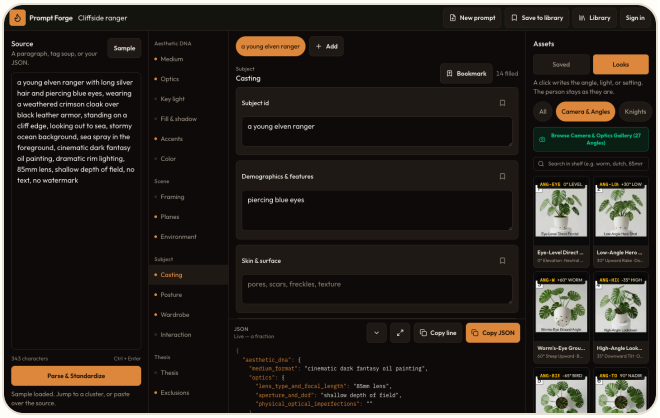
PROMPT FIELD	EXAMPLE VALUE
`framing_doctrine.named_principle`	"Worm's-eye dramatic ground rake"
`framing_doctrine.camera_elevation_and_axis`	"camera resting directly on floor plane, tilted steeply upward at 60°…"
`optics.lens_type_and_focal_length`	"24mm wide angle" (only on cards that specify a lens)
`optics.aperture_and_dof`	"f/8.0 deep ground-to-canopy depth"



The Camera & Optics Reference Gallery: badges, category filters with counts, search, and a live preview of the text each card writes.



Click “Worm’s-Eye”: Framing and the JSON update instantly.



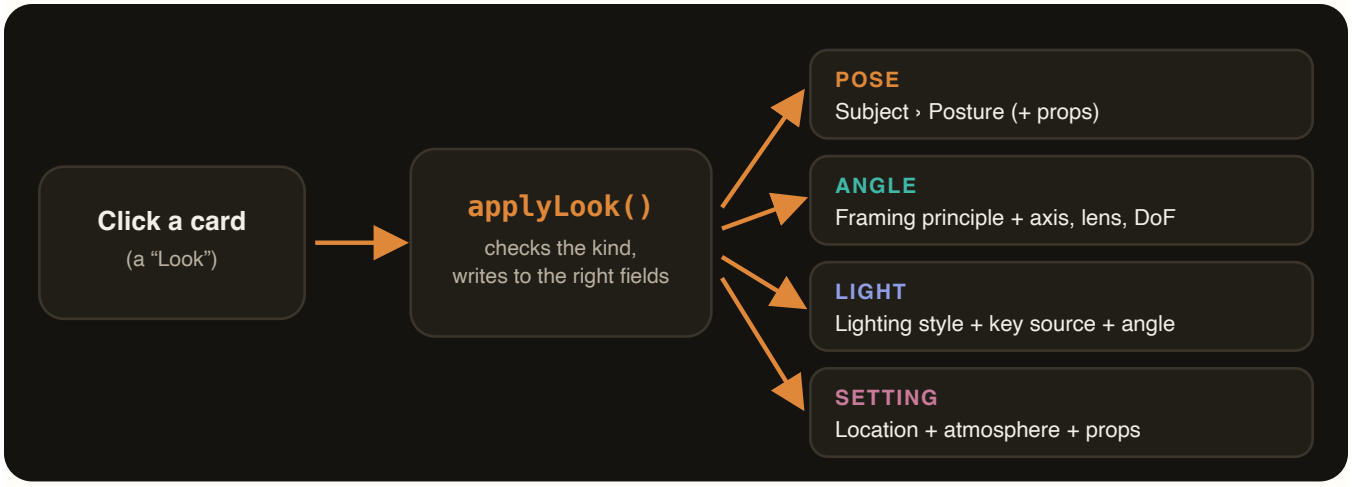
The same cards also live in the right-hand Looks shelf under “Camera & Angles.”

CHAPTER 10

The “Looks” engine

One tiny function that makes every card in the app work.

Poses, camera angles, lights and settings look different but behave the same way: *click, then write some text into specific fields.* So we gave them one shared shape, called a **Look**, and one function, `applyLook()`, that knows where each kind writes.



Four kinds of card, one function.

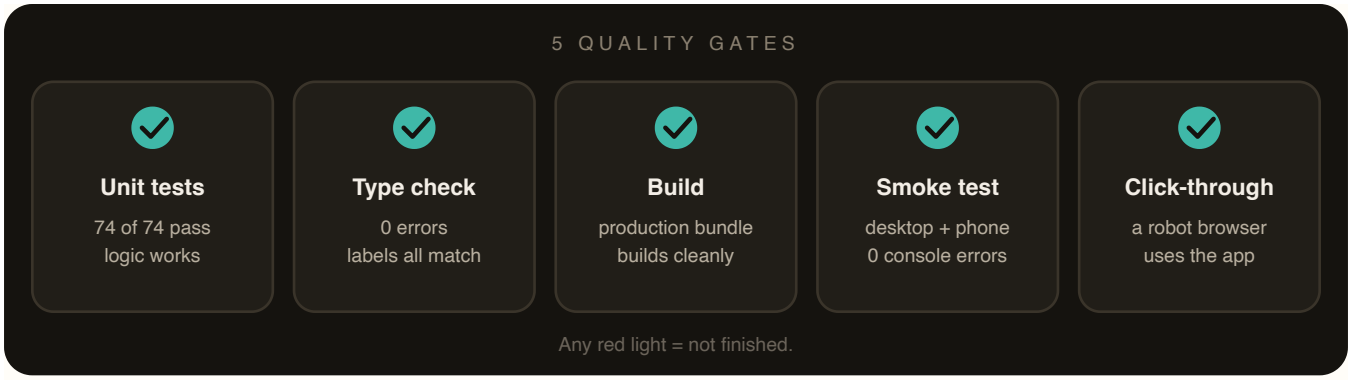
What the shelf shows

- **Filters:** All · Camera & Angles · Knights · Photoshoot · Fine Art · Cinematic · Couples & Duos · Action · Sitting · Floor · All Poses · Light · Setting.
- **About 160 looks in total:** 117 poses, 33 angles, 5 lights and 3 settings. The poses include the 97 new ones plus older entries kept so existing prompts still work.
- **Applied marker:** `lookIsApplied()` compares your current prompt with each card, so the card currently in use shows a check mark.
- **Content gating:** a few legacy poses are flagged `adult`. They’re excluded from every normal shelf and appear only under their own filter. A test guards that they never leak into the modeling shelf.

CHAPTER 11

How we know it works

Five checks, run before every “done.”



GATE	COMMAND	WHAT IT PROTECTS
Unit tests	`npm test`	The brain: parser, bookmarks, Looks. Includes tests that angles write to the right fields and that at least 20 camera angles exist.
Type check	`npm run typecheck`	Catches mismatched data (like passing a number where text is expected) before they become bugs.
Build	`npm run build`	Makes sure the real, deployable version compiles. This is what goes to the internet.
Smoke test	`node scripts/browser-smoke.mjs`	Opens the app in a headless browser at desktop (1280×800) and phone (390×844) sizes, checks content shows, and the console is clean.
Click-through	Playwright script	A scripted browser opens the gallery, filters, clicks a card, and confirms the prompt JSON changed.

JARGON CHECK · SMOKE TEST

Borrowed from electronics: plug it in and see if smoke comes out. If the page loads and nothing explodes, it passes the first, most basic test.

STATUS NOTE

The tests and type check were re-run for this guide and pass (74/74, 0 errors). The browser-based smoke test passed during development, but when I tried to re-run it while preparing this document, macOS blocked the headless browser from launching, so that one wasn't repeated today.

NEXT STEPS

Rough edges & roadmap

Known rough edges (worth fixing)

ISSUE	DETAIL
Numbers burned into pose photos	The big digits (“5”, “2”, “7”) in the corners of many pose cards are leftovers from the contact-sheet grids. Re-cropping a little tighter, or regenerating without numbers, fixes it.
Captions burned into camera photos	Each plant card has its own text (“Eye-Level Direct Frontal”) baked into the picture, which duplicates the title underneath. A few cards also show a small corner artifact from the grid.
Clipped ID badges in the sidebar	In the narrow right shelf, IDs like `ANG–EYE…` are cut off. Shortening or hiding the ID there is a quick CSS tweak.
Leftover stick-figure data	The old drawing file `figure.ts` is still in the code. It no longer appears as cards, and can be deleted in a cleanup pass.
Small pose collections	Sitting (5), Casual (4), Fantasy (3), Floor (3) are thin compared with the 18-pose collections.

Roadmap

Option 2 · Visual Wardrobe & Outfit Catalog

The same treatment for clothing: reference sheets for knight plate armor, techwear, editorial fashion, historical regalia and fantasy robes. Each card would write into the Wardrobe slot (*layers, textiles, hardware, wear & patina, footwear*). Modeled on the outfit reference image you shared.

Option 3 · Visual Lighting & Atmosphere Studio

Lighting cards on a controlled subject: Rembrandt, chiaroscuro, neon rim, golden hour, volumetric fog. Writing into Key light, Fill, Accents and Environment.

Bigger ideas, parked for later

- **Subject-specific pose filters** (poses that only suit knights, or only portraits).
- **Growing the pose library toward hundreds**, prioritizing the thin collections.
- **Regenerating the numbered/captioned images** cleanly with the image tool once its daily quota resets.

Run it yourself

```
# You need Node 24 (this project uses a newer Node feature)
npm install
npm run dev          # starts the app at http://localhost:8080
npm test             # 74 tests
npm run typecheck    # the label-checker
npm run build        # make the production version
```

On this Mac, Node 24 is installed at `~/.nvm/versions/node/v24.11.1/bin`. The default system Node 18 is too old and will fail with an error mentioning `styleText`.

REFERENCE

Glossary

Every technical word in this guide, in one place.

WORD	MEANING
Aperture / f-stop	How wide the lens opens. Small f-number (f/1.4) = blurry background; large (f/16) = everything sharp.
authMiddleware	A checkpoint that confirms who's signed in before touching saved data.
Blob	A raw chunk of file data, like a photo.
Bokeh	The soft blur of out-of-focus areas.
Build	Packaging the code into the final optimized files that get deployed.
Cluster	One of the 15 prompt slots in the left-hand menu.
Component	A reusable piece of screen (a card, a button, a whole panel) in React.
Contact sheet	One image holding a grid of many small images; we use 3×3 sheets.
Dutch angle	A deliberately tilted camera that makes the horizon slant.
Depth of field (DoF)	How much of the scene, near to far, is in focus.
Focal length	The lens "zoom" in millimeters. Small = wide, large = tight.
Frontend / backend	What runs in your browser vs. what runs on a server.
HMR	Hot Module Replacement: the page updates live as code is edited.
IndexedDB	A larger database built into browsers, used here for photos.
JSON	A labeled, nested text format: `"label": "value"`.
localStorage	A small key-value notepad every browser provides (about 5 MB).
Look	Any clickable card (pose, angle, light, setting) that writes into the prompt.
Migration	A script that creates or changes database tables.
Modal	A full-screen or pop-up panel that sits on top of the page (the galleries).
Nadir	Straight down. A "nadir shot" points the camera at the ground.

Parse	Read text and organize it into structure.
PostgreSQL	A popular, reliable database.
Prompt	The written instructions you give an AI image generator.
Regex	A mini language for matching patterns in text.
Route	A page address, like `/` or `/login`.
Server function	Code that runs on the server but can be called straight from the app.
Smoke test	The quickest “does it even turn on?” check.
Stance	The pose’s basic body position: standing, sitting, kneeling, etc.
Tailwind	A styling system that uses small class names on elements.
Type / TypeScript	Labels describing what kind of data each value is, checked automatically.
Unit test	A small automatic check of one piece of logic.
Vite	The tool that serves the app while developing and bundles it for release.